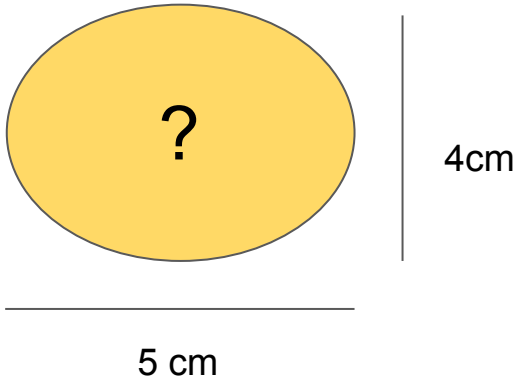
¿Puedes predecir a qué fruta corresponde este valor?



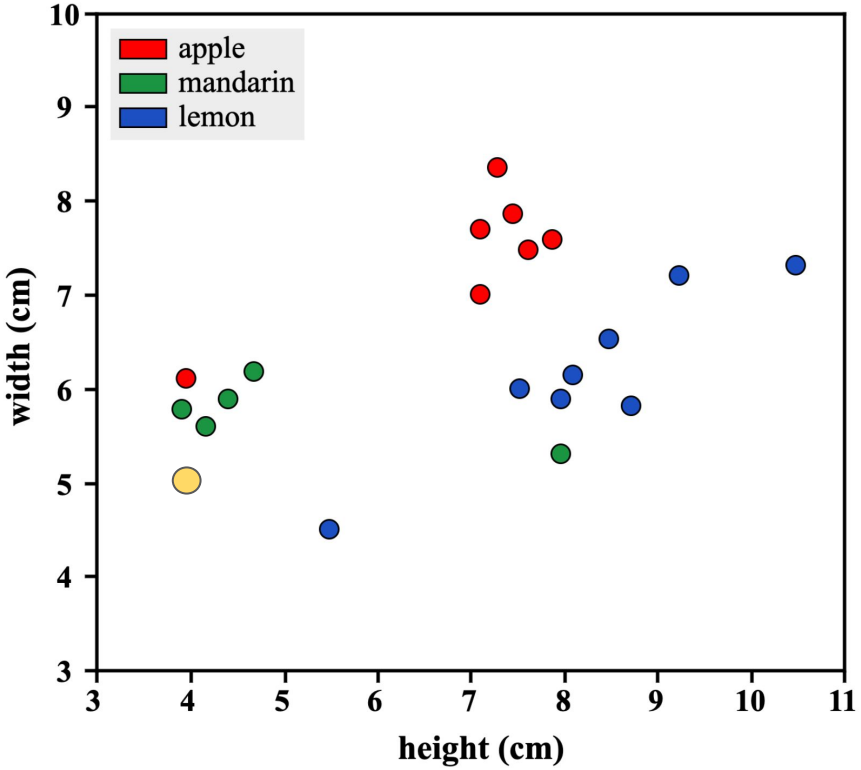
height (cm)	width (cm)	**Fruit Type**
3.91	5.76	mandarin
7.09	7.69	apple
10.48	7.32	lemon
9.21	7.20	lemon
7.95	5.90	lemon
7.62	7.51	apple
7.95	5.32	mandarin
4.69	6.19	mandarin
7.50	5.99	lemon
7.11	7.02	apple
4.15	5.60	mandarin
7.29	8.38	apple
8.49	6.52	lemon
7.44	7.89	apple
7.86	7.60	apple
3.93	6.12	apple
4.40	5.90	mandarin
5.5	4.5	lemon
8.10	6.15	lemon
8.69	5.82	lemon

knn nearest neighbor

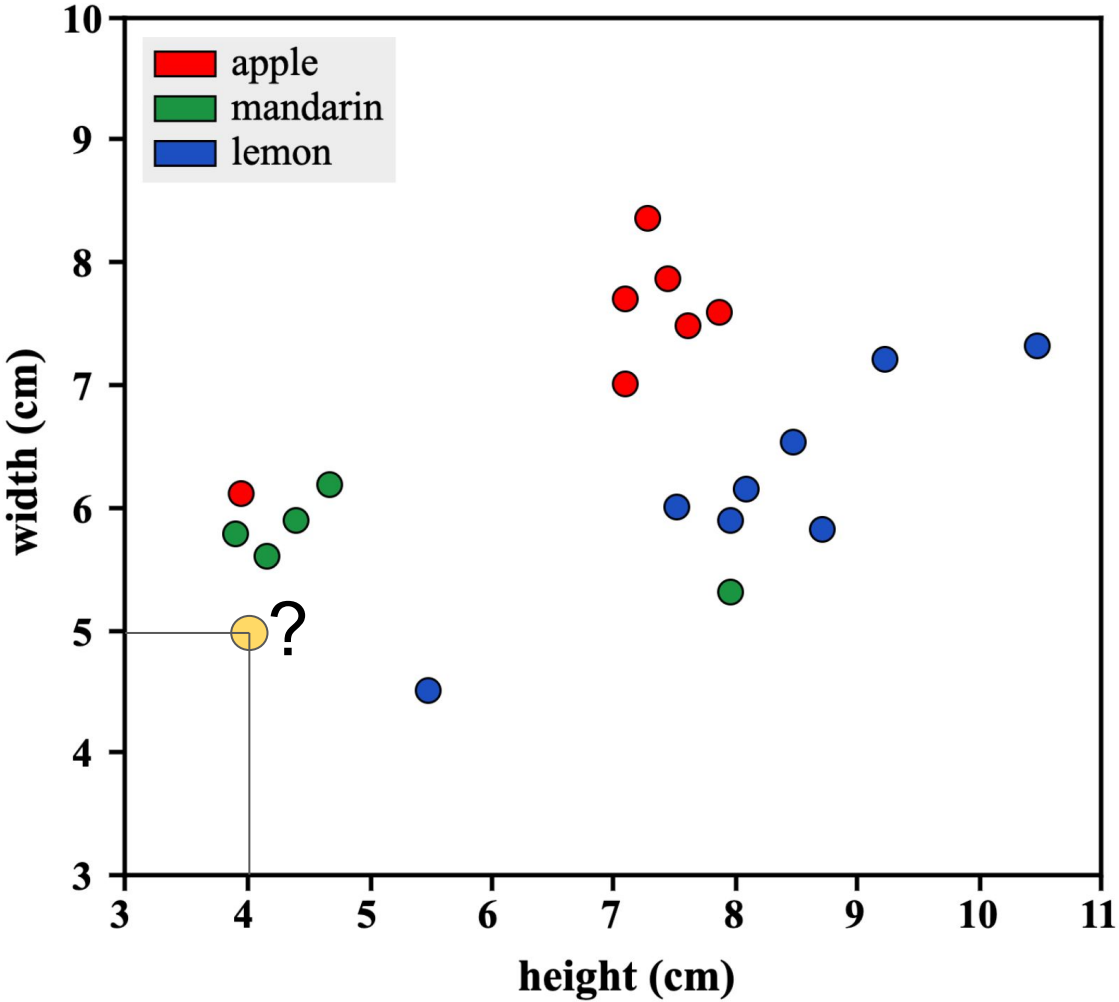
Evelyn Fix and Joseph Hodges in 1951

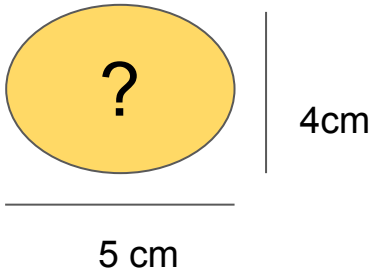
Algoritmo vecinos más cercanos

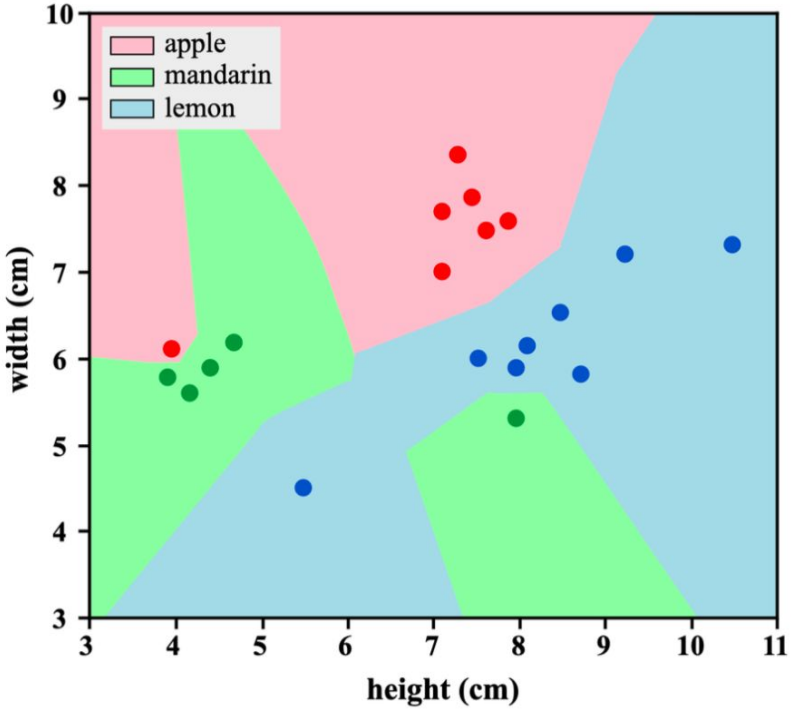
- Clasificador de **aprendizaje supervisado**
- Utiliza la **proximidad** para hacer clasificaciones o predicciones
- Regresión o clasificación
- Se asigna una etiqueta por **voto mayoritario**



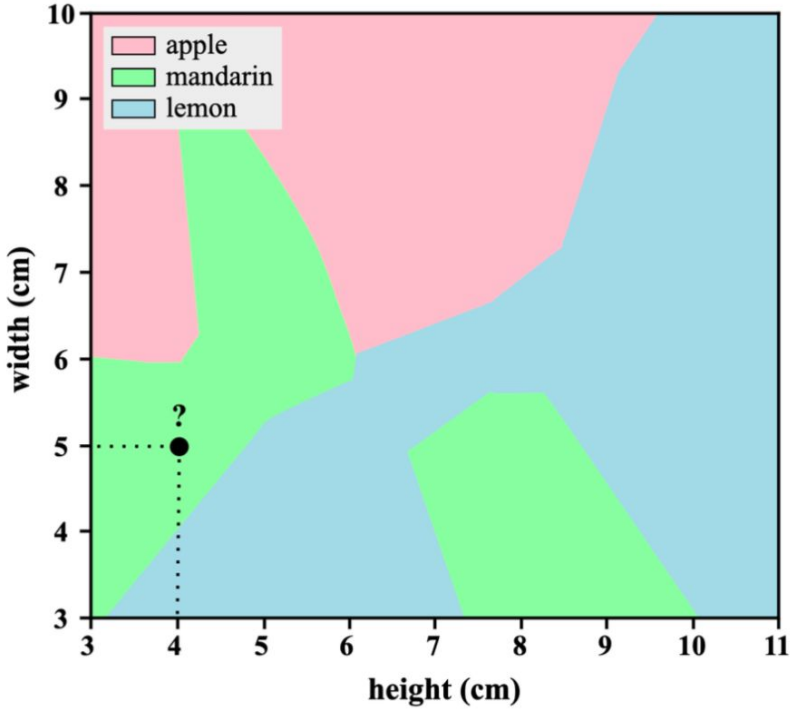
- Se selecciona un valor k, que representa el número de vecinos más cercanos que se utilizarán para hacer una predicción.
- Se miden las distancias entre el nuevo punto de datos y cada punto en el conjunto de entrenamiento.
- Se seleccionan los k-vecinos más cercanos basados en las distancias calculadas.
- Se asigna una etiqueta o valor a este nuevo punto de datos basándose en la mayoría de las etiquetas de los k vecinos más cercanos.
-


apple
mandarin
lemon
?
width (cm)
height (cm)
10
9
8
7
6
5
4
3
3
4
5
6
7
8
9
10
11


?
4cm
5 cm



(a) Learned decision regions (and training data)



(b) Classify new point

scikit-learn syntax

```
from sklearn.module import Model
model = Model()
model.fit(X, y)
predictions = model.predict(X_new)
print(predictions)
```

```
array([0, 0, 0, 0, 1, 0])
```

Using scikit-learn to fit a classifier

```
from sklearn.neighbors import KNeighborsClassifier
X = churn_df[["total_day_charge", "total_eve_charge"]].values
y = churn_df["churn"].values
print(X.shape, y.shape)
```

```
(3333, 2), (3333,)
```

```
knn = KNeighborsClassifier(n_neighbors=15)
knn.fit(X, y)
```

Predicting on unlabeled data

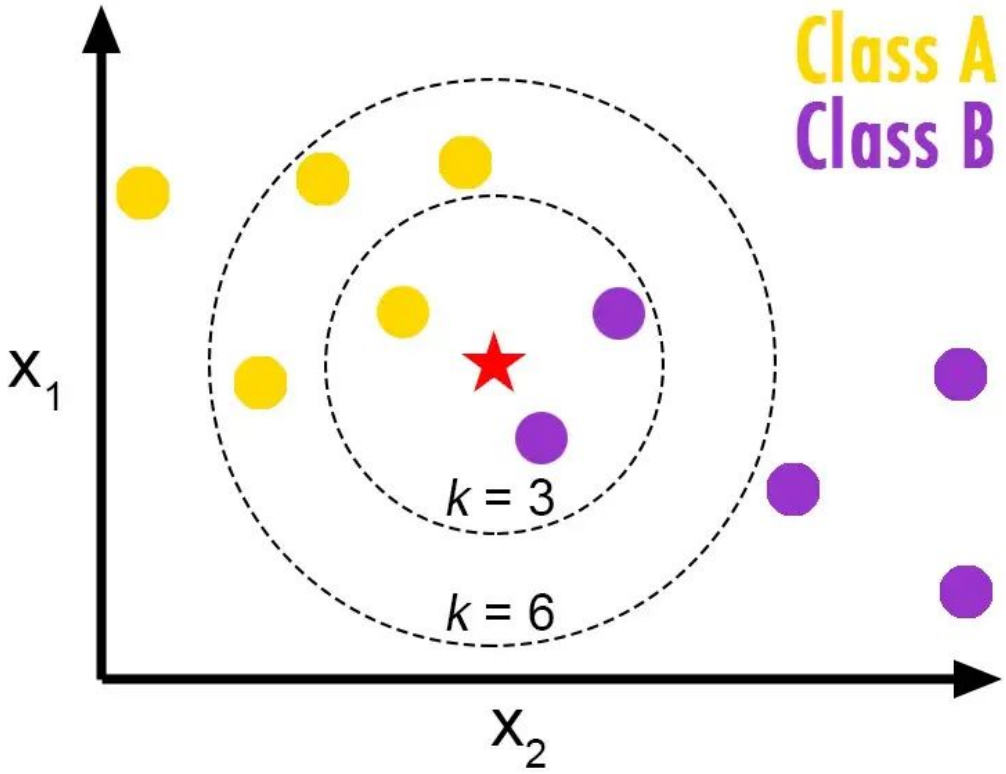
```
X_new = np.array([[56.8, 17.5],
                  [24.4, 24.1],
                  [50.1, 10.9]])
print(X_new.shape)
```

```
(3, 2)
```

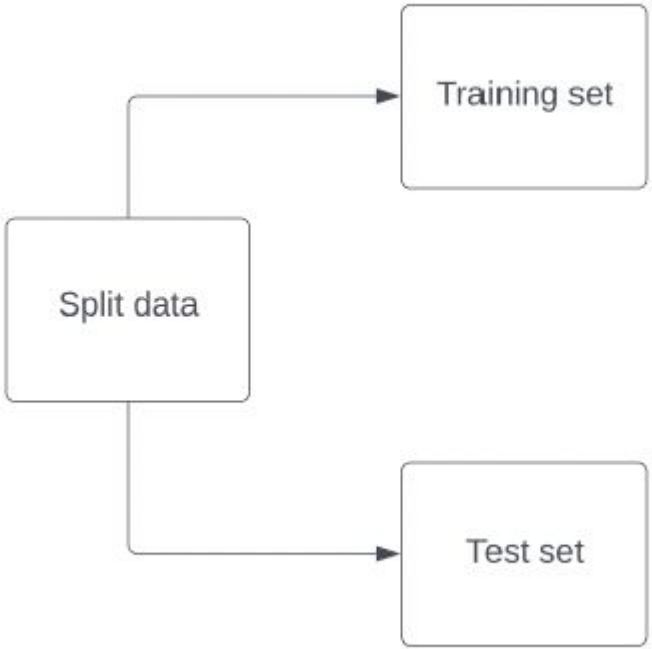
```
predictions = knn.predict(X_new)
print('Predictions: {}'.format(predictions))
```

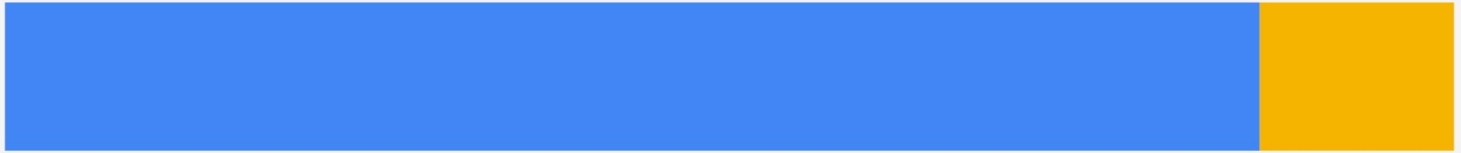
```
Predictions: [1 0 0]
```

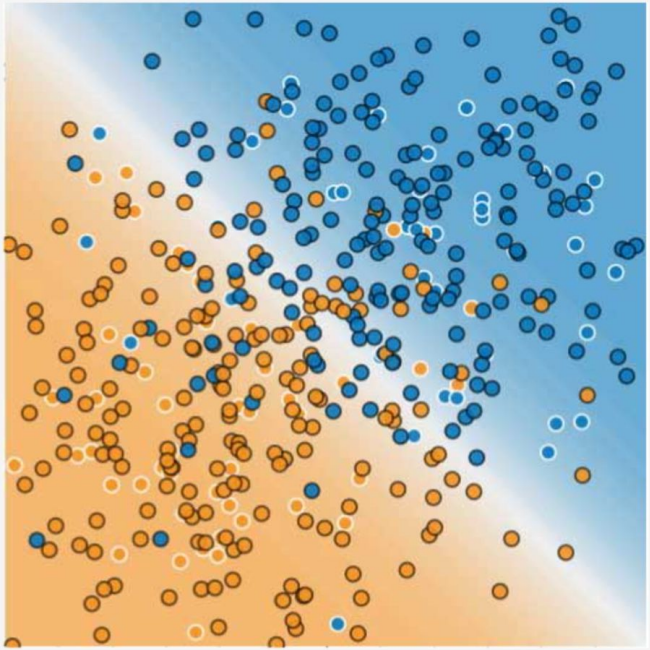
Measuring model performance



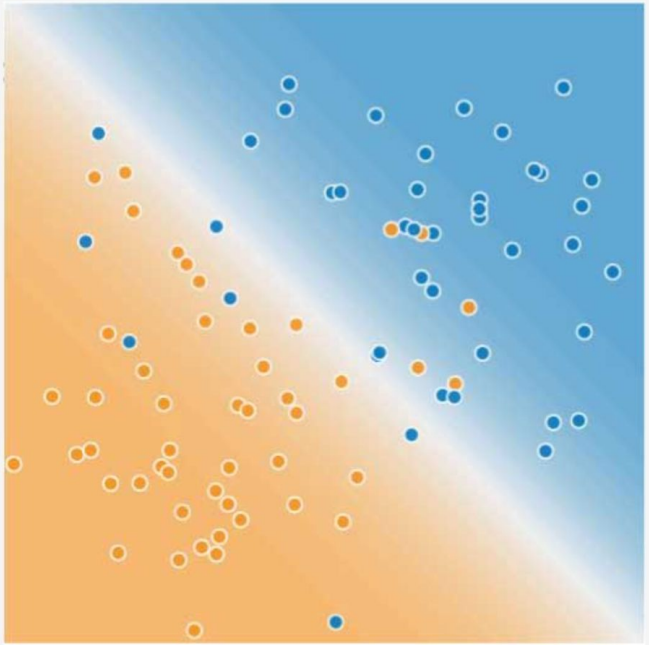
Computing accuracy



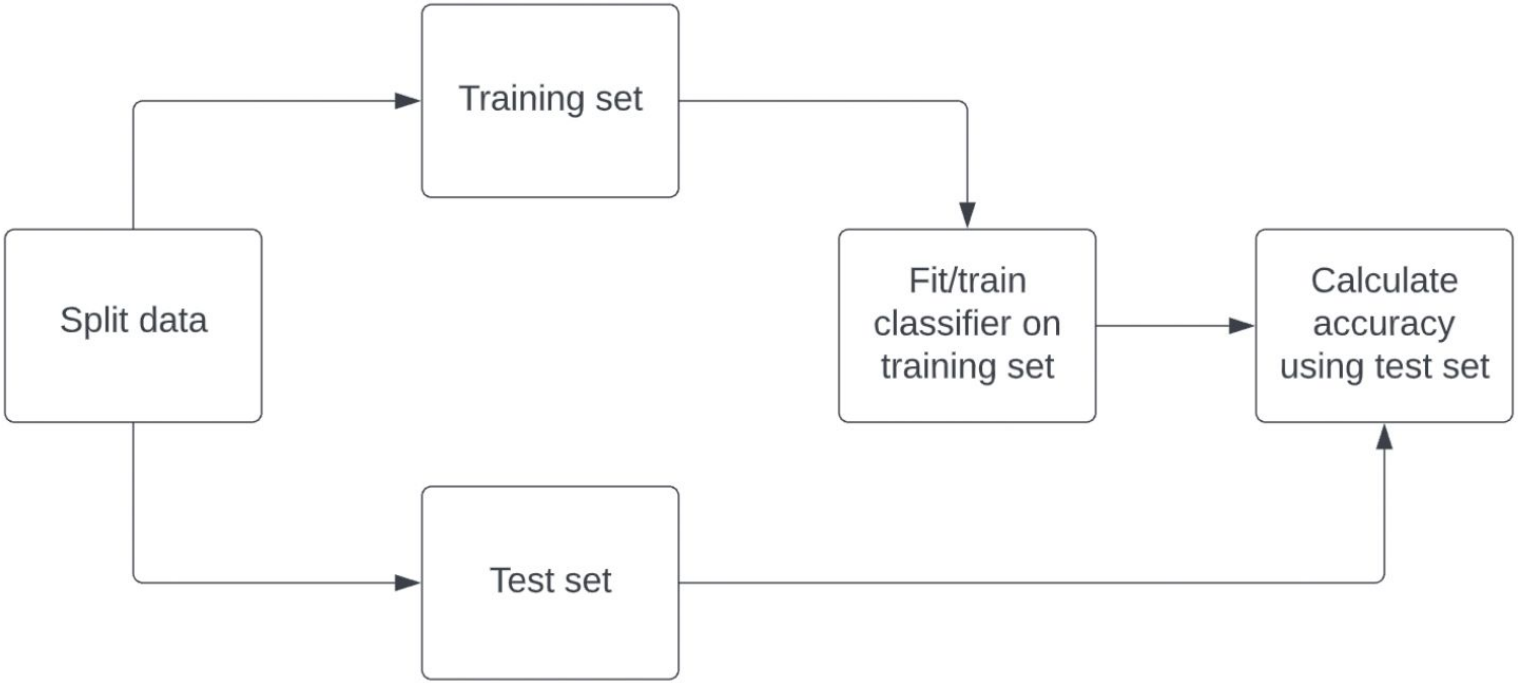




Training Data

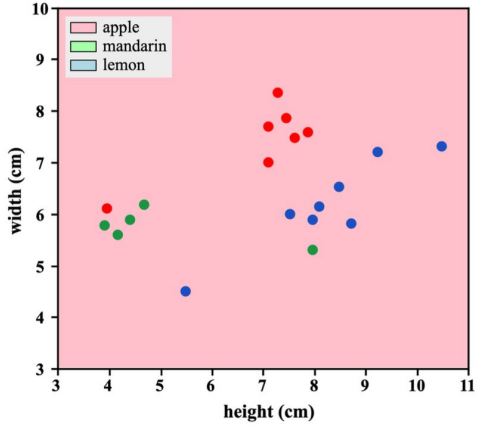


Test Data

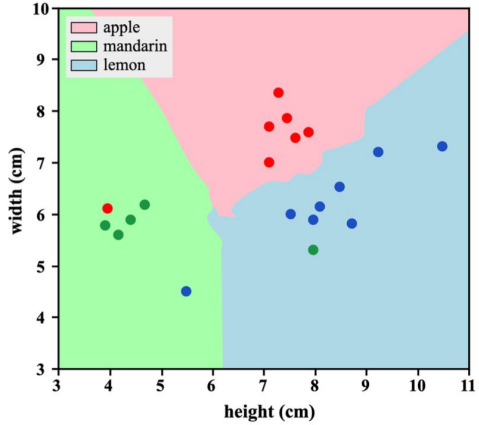

Training set
Split data
Fit/train classifier on training set
Calculate accuracy using test set
Test set

Train test split

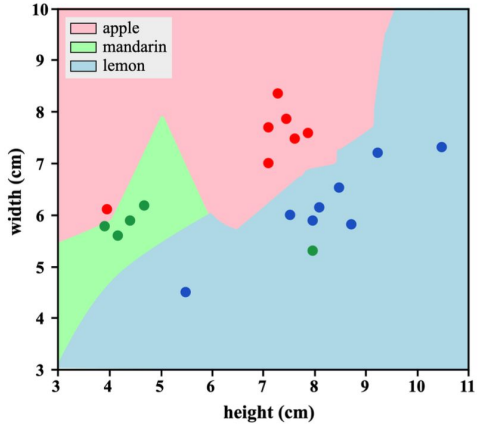
```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
                                                    random_state=21, stratify=y)
knn = KNeighborsClassifier(n_neighbors=6)
knn.fit(X_train, y_train)
print(knn.score(X_test, y_test))
```



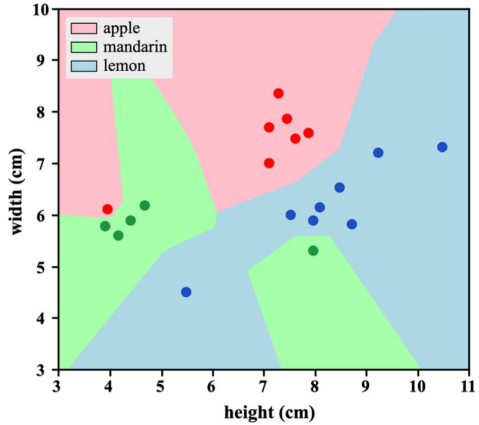
(a) $k = n = 20$ (severely under-fit)



(b) $k = 5$ (optimal fit)



(c) $k = 2$ (mildly overfit)



(d) $k = 1$ (severely over-fit)

Larger k = less complex model = can cause underfitting

Smaller k = more complex model = can lead to overfitting